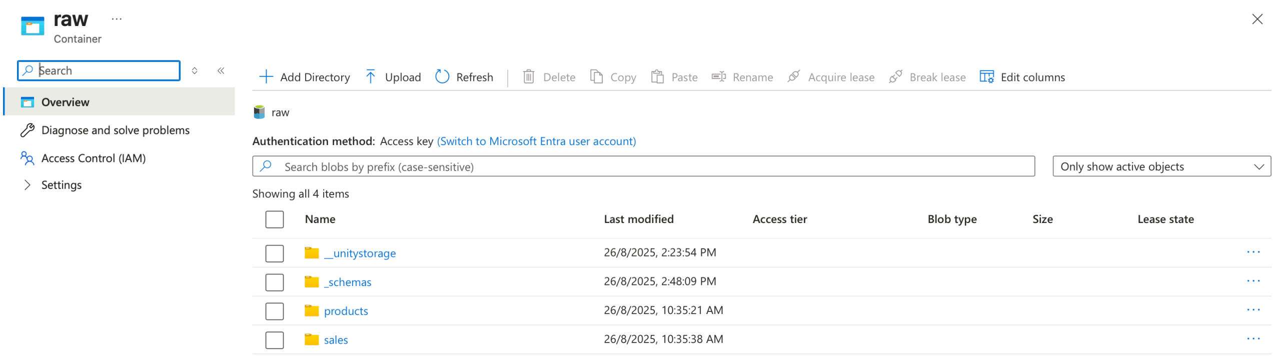
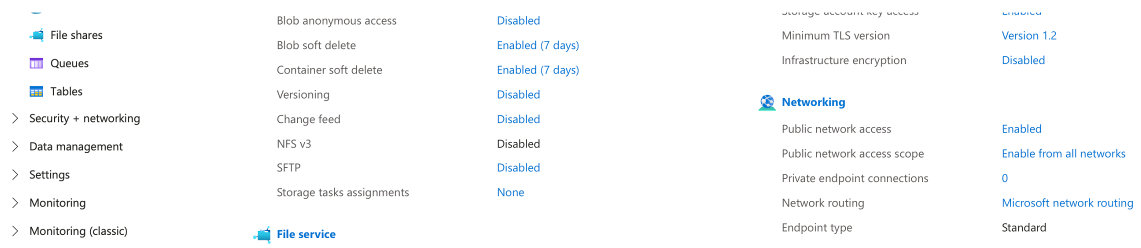
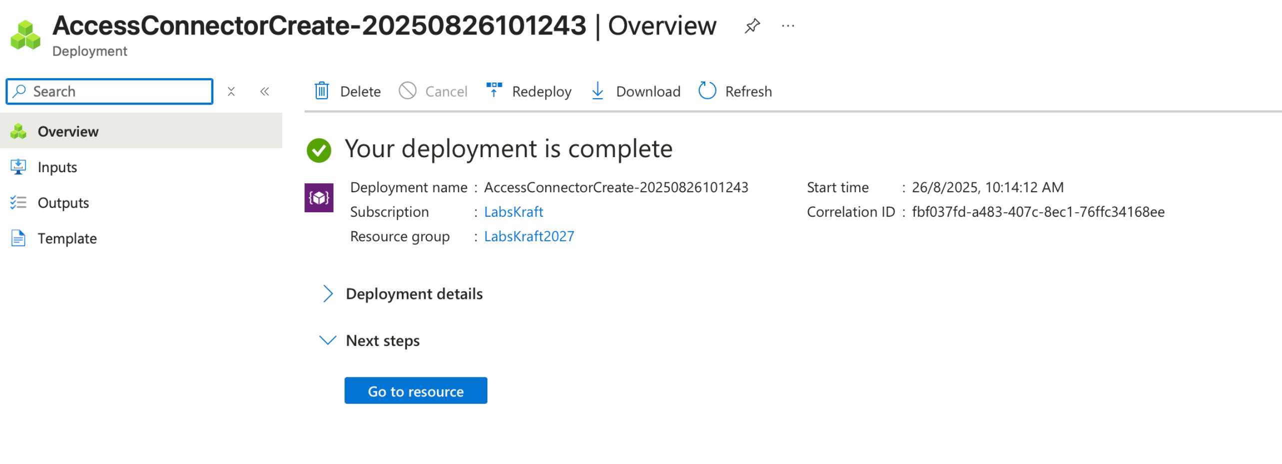
Shivankur Ghaie
Capstone Project 2

1. Creation of storage account:

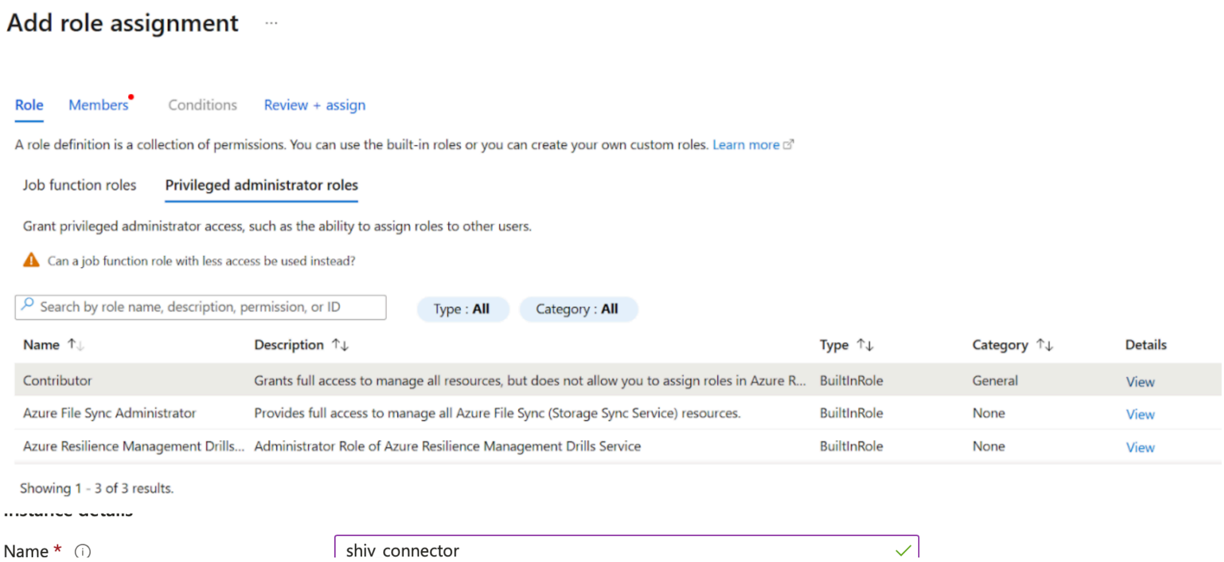


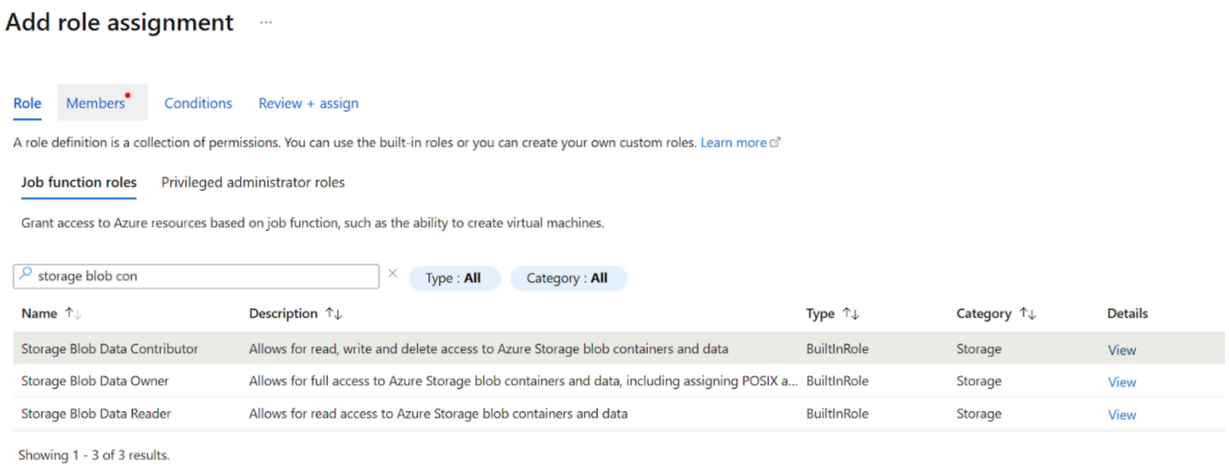


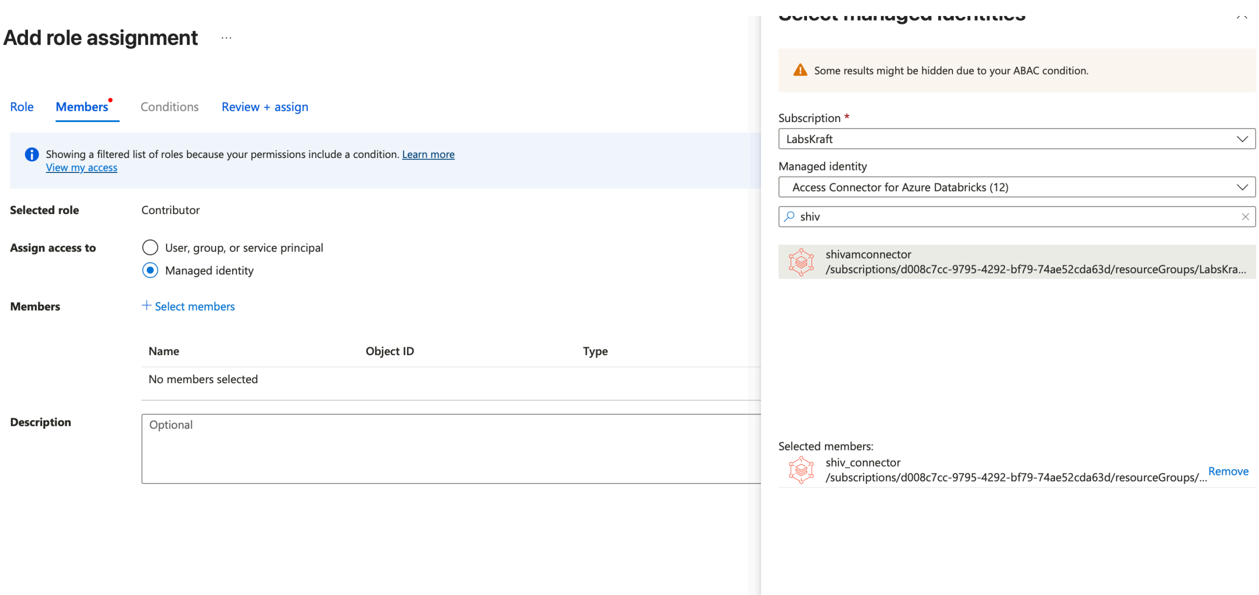
2. Creation of access connector:



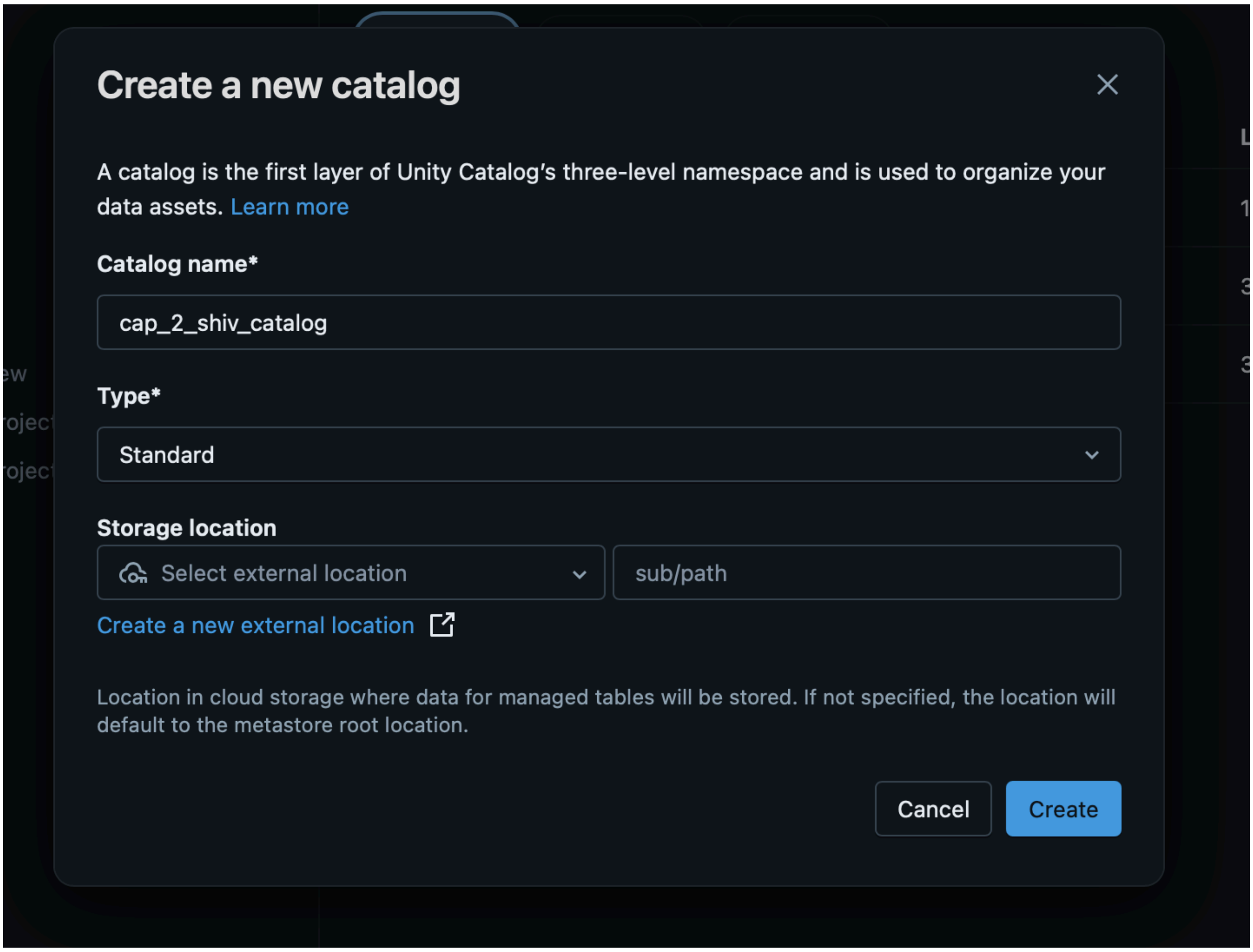
3. Giving access connector permission for storage:

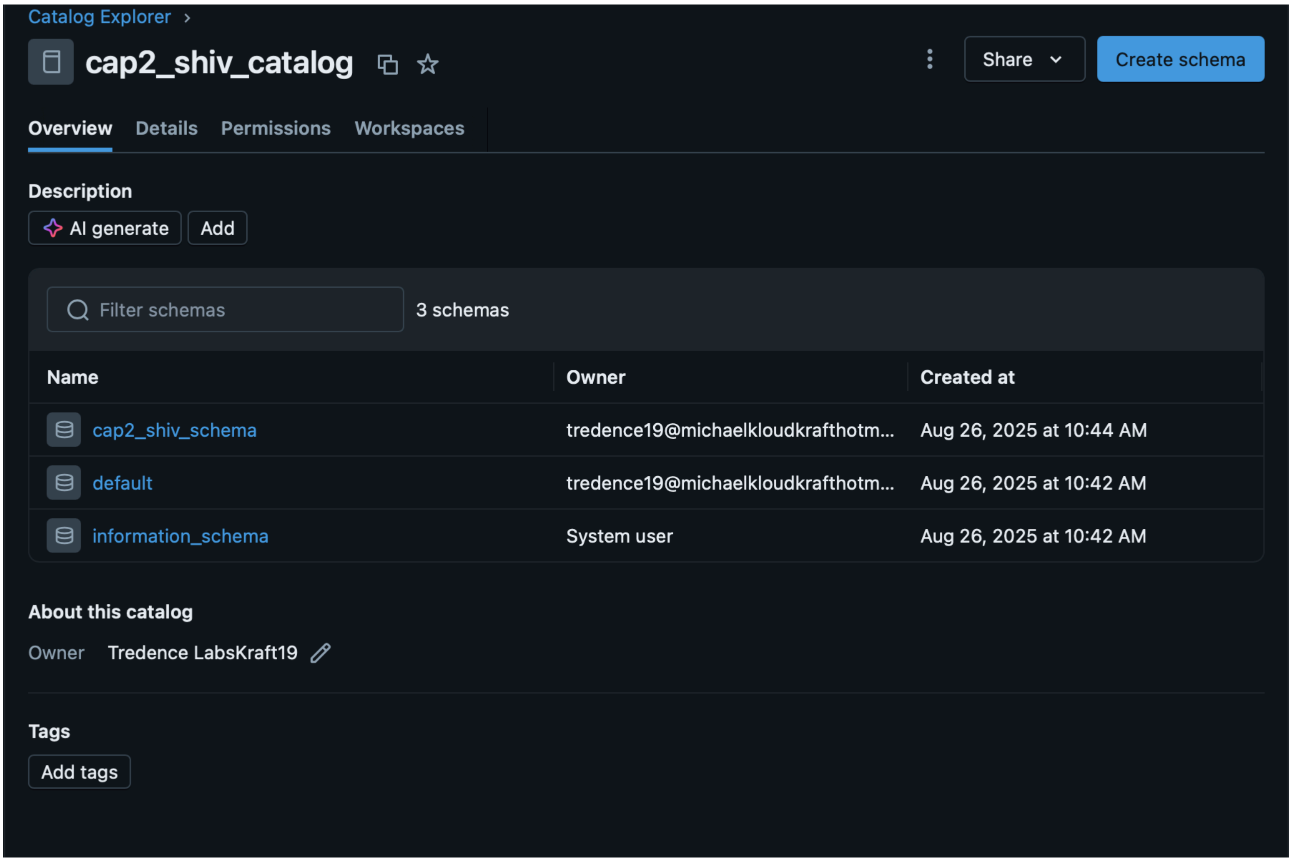




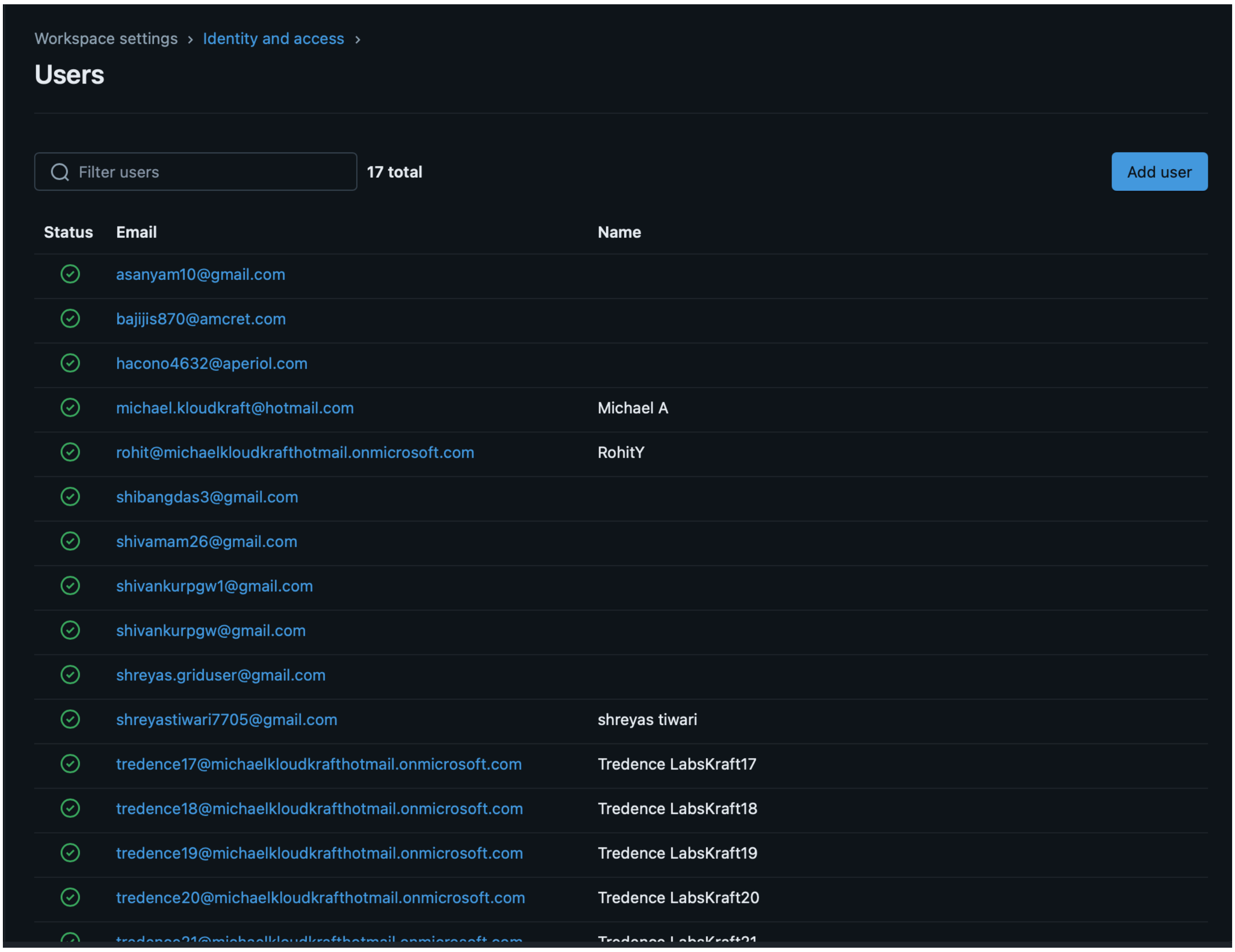


4. Creation of Unity Catalog:

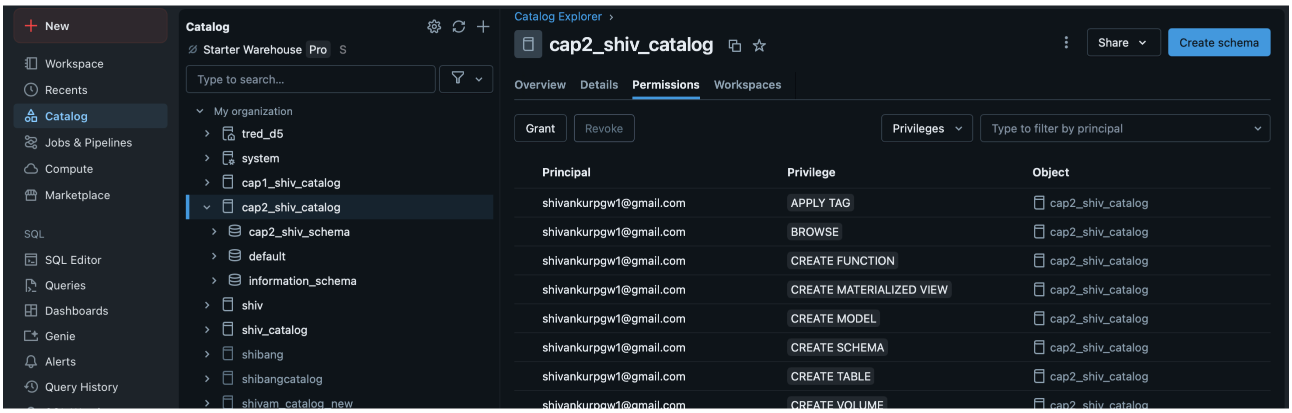




5. Adding User to Unity Catalog



6. Granting Access to Users



Ingestion

1. Schema Creation:

```
%sql
USE CATALOG cap2_shiv_catalog;

%sql
CREATE SCHEMA IF NOT EXISTS cap2_shiv_schema
COMMENT 'Capstone 2 Schema';

%sql
CREATE TABLE cap2_shiv_catalog.cap2_shiv_schema.ext_sales_raw
USING CSV
OPTIONS (
  path 'abfss://raw@shivghaie.dfs.core.windows.net/sales/',
  header 'true',
  inferSchema 'true'
);
```

2 minutes ago (2m) 4 SQL

```
%sql
SELECT * FROM cap2_shiv_catalog.cap2_shiv_schema.ext_sales_raw LIMIT 10;
```

(1) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [sale_id: integer, product_id: integer ... 5 more fields]

Table

	sale_id	product_id	region	store_id	quantity	sales_amount	sale_date
1	1	157	APAC	Store_31	16	1289.03	2025-08-11T21:20:34.000+00:00
2	2	65	APAC	Store_9	8	1301.48	2025-08-12T02:40:42.000+00:00
3	3	416	LATAM	Store_25	19	3047.56	2025-08-10T00:09:00.000+00:00
4	4	501	US	Store_29	18	1955.74	2025-07-28T09:49:56.000+00:00
5	5	244	MEA	Store_6	20	2362.98	2025-08-15T00:28:03.000+00:00
6	6	240	EU	Store_22	8	1516.96	2025-08-25T05:58:18.000+00:00
7	7	854	EU	Store_24	10	178.92	2025-07-28T09:55:39.000+00:00
8	8	69	US	Store_9	7	246.4	2025-08-10T22:41:18.000+00:00
9	9	654	EU	Store_3	14	138.23	2025-08-02T05:50:22.000+00:00

```
%sql
CREATE TABLE cap2_shiv_catalog.cap2_shiv_schema.ext_products
USING JSON
OPTIONS (
  path 'abfss://raw@shivghaie.dfs.core.windows.net/products/',
  multiline 'false'
);
```

Medallion Architecture

(i) Bronze Layer

```
import dlt
from pyspark.sql.functions import *
# Ingest raw sales data (multiple daily CSV files in raw/sales/)
@dlt.table(
  name="sales_raw",
  comment="Raw sales data ingested from CSV files",
  table_properties={"quality": "bronze"}
)
def sales_raw():
    return (
        spark.readStream.format("cloudFiles")     # Auto Loader for incremental loads
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .load("abfss://raw@shivghaie.dfs.core.windows.net/sales/")
    )
# Products CSV
@dlt.table(
  name="pro_csv",
  comment="Raw product data from CSV",
  table_properties={"quality": "bronze"}
)
def pro_csv():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .load("abfss://raw@shivghaie.dfs.core.windows.net/products/")
    )
# Products JSON
@dlt.table(
  name="pro_json",
  comment="Raw product data from JSON",
  table_properties={"quality": "bronze"}
)
def pro_json():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .load("abfss://raw@shivghaie.dfs.core.windows.net/products/")
    )
# Unified Products (CSV + JSON)
@dlt.view(name="products")
def products():
    return dlt.read("pro_csv").unionByName(dlt.read("pro_json"), allowMissingColumns=True)
# Clean Sales
@dlt.table(
```

```
  name="sales_clean",
  comment="Cleaned sales data with enforced schema",
  table_properties={"quality": "silver"}
)
```

(ii) Silver Layer

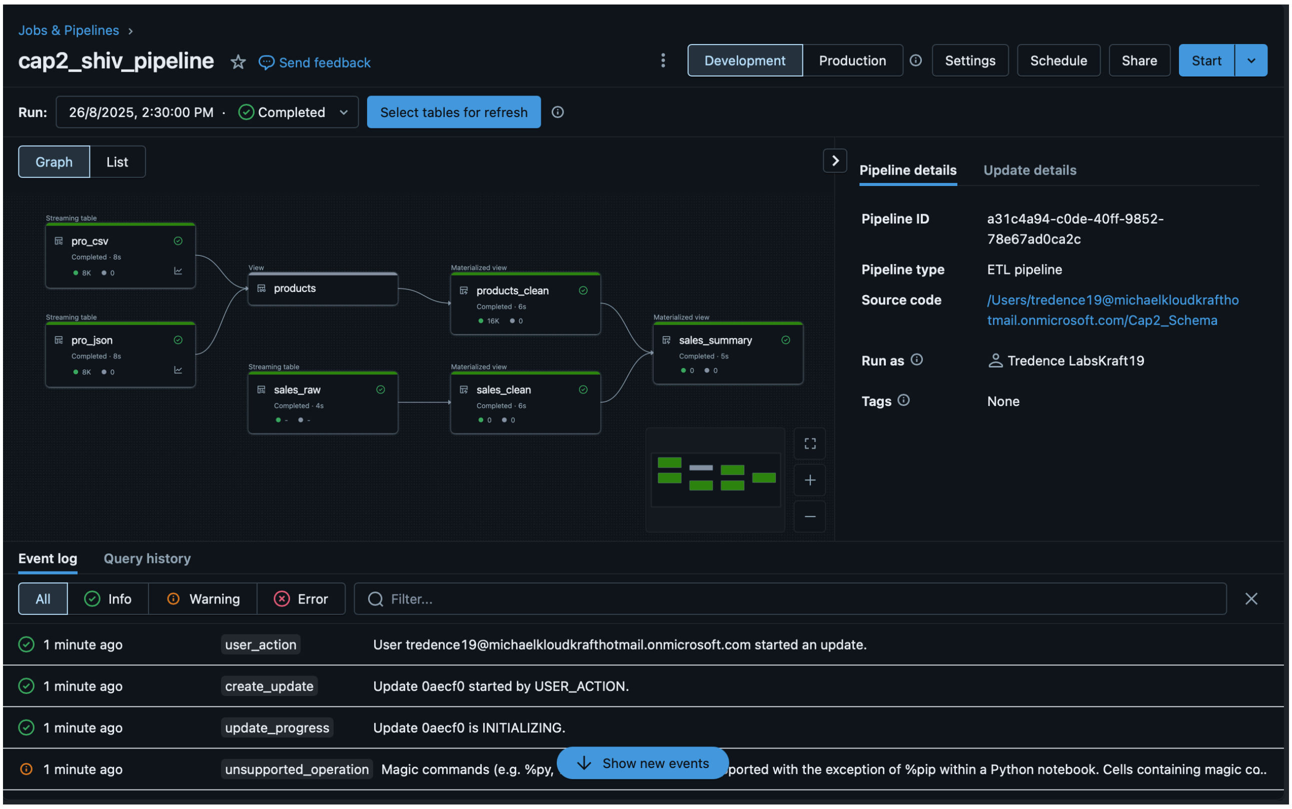
```
@dlt.expect("valid_sales_amount", "sales_amount >= 0")
@dlt.expect("valid_quantity", "quantity > 0")
def sales_clean():
    return (
        dlt.read("sales_raw")
        .withColumn("sale_id", col("sale_id").cast("string"))
        .withColumn("product_id", col("product_id").cast("string"))
        .withColumn("region", col("region").cast("string"))
        .withColumn("store_id", col("store_id").cast("string"))
        .withColumn("quantity", col("quantity").cast("int"))
        .withColumn("sales_amount", col("sales_amount").cast("double"))
        .withColumn("sale_date", to_date(col("sale_date"), "yyyy-MM-dd"))
        .na.drop()
    )
@dlt.table(
  name="products_clean",
  comment="Cleaned products data with schema evolution support",
  table_properties={"quality": "silver"}
)
def products_clean():
    return (
        dlt.read("products")
        .withColumn("product_id", col("product_id").cast("string"))
        .withColumn("product_name", col("product_name").cast("string"))
        .withColumn("category", col("category").cast("string"))
        .withColumn("price", col("price").cast("double"))
        .withColumn("discount_rate", col("discount_rate").cast("double"))
    )
```

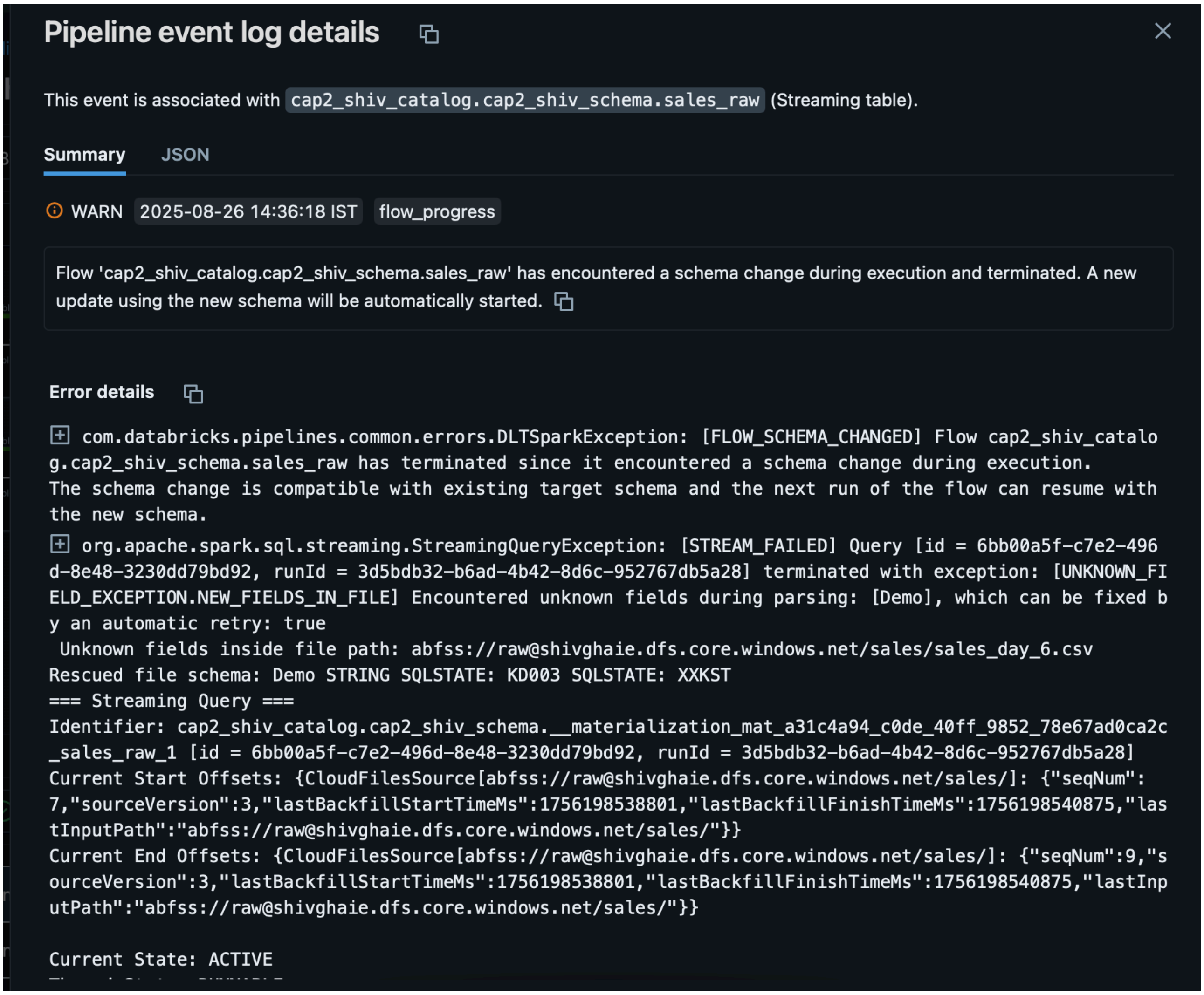
(iii) Gold Layer

```
# Join sales + products and calculate KPIs
@dlt.table(
  name="sales_summary",
  comment="Daily sales summary by product and region",
  table_properties={"quality": "gold"}
)
def sales_summary():
    sales = dlt.read("sales_clean")
    products = dlt.read("products_clean")
    return (
        sales.join(products, "product_id", "left")
        .groupBy("region", "sale_date", "product_id", "product_name", "category")
```

```
        .agg(
            sum("sales_amount").alias("daily_revenue"),
            sum("quantity").alias("units_sold"),
            avg("price").alias("avg_price"),
            avg("discount_rate").alias("avg_discount")
        )
    )
```

Final Output





Pipeline event log details

This event is associated with `cap2_shiv_catalog.cap2_shiv_schema.sales_raw` (Streaming table).

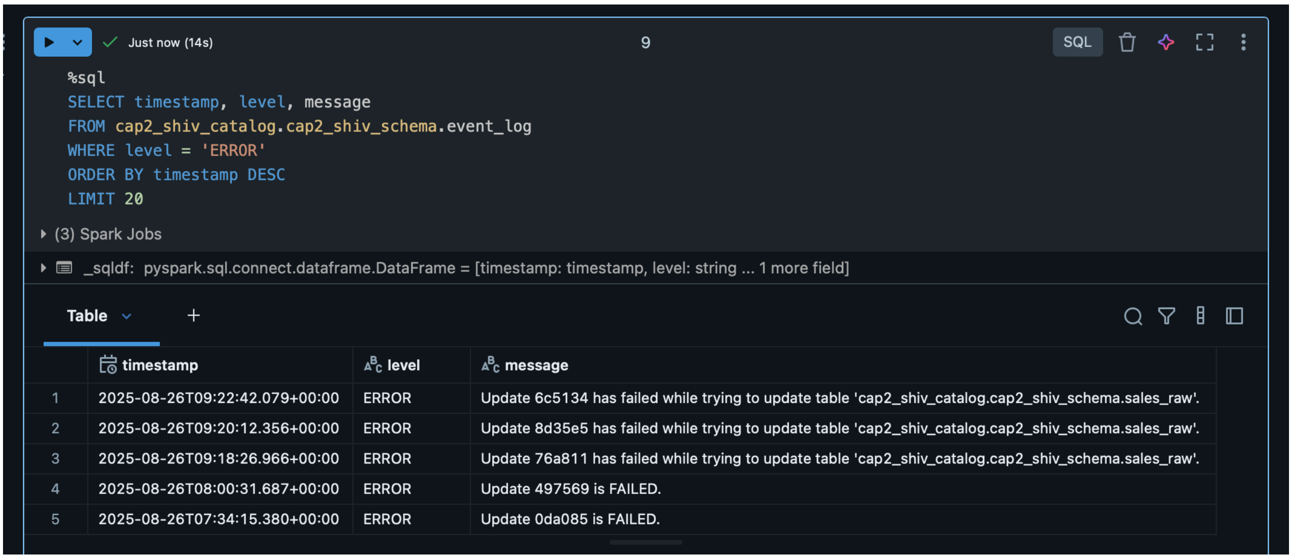
Summary JSON

WARN 2025-08-26 14:36:18 IST flow_progress

Flow 'cap2_shiv_catalog.cap2_shiv_schema.sales_raw' has encountered a schema change during execution and terminated. A new update using the new schema will be automatically started.

Error details

```
com.databricks.pipelines.common.errors.DLTSparkException: [FLOW_SCHEMA_CHANGED] Flow cap2_shiv_catalo
g.cap2_shiv_schema.sales_raw has terminated since it encountered a schema change during execution.
The schema change is compatible with existing target schema and the next run of the flow can resume with
the new schema.
org.apache.spark.sql.streaming.StreamingQueryException: [STREAM_FAILED] Query [id = 6bb00a5f-c7e2-496
d-8e48-3230dd79bd92, runId = 3d5bdb32-b6ad-4b42-8d6c-952767db5a28] terminated with exception: [UNKNOWN_FI
ELD_EXCEPTION.NEW_FIELDS_IN_FILE] Encountered unknown fields during parsing: [Demo], which can be fixed b
y an automatic retry: true
 Unknown fields inside file path: abfss://raw@shivghaie.dfs.core.windows.net/sales/sales_day_6.csv
Rescued file schema: Demo STRING SQLSTATE: KD003 SQLSTATE: XXKST
=== Streaming Query ===
Identifier: cap2_shiv_catalog.cap2_shiv_schema.__materialization_mat_a31c4a94_c0de_40ff_9852_78e67ad0ca2c
_sales_raw_1 [id = 6bb00a5f-c7e2-496d-8e48-3230dd79bd92, runId = 3d5bdb32-b6ad-4b42-8d6c-952767db5a28]
Current Start Offsets: {CloudFilesSource[abfss://raw@shivghaie.dfs.core.windows.net/sales/]: {"seqNum":
7,"sourceVersion":3,"lastBackfillStartTimeMs":1756198538801,"lastBackfillFinishTimeMs":1756198540875,"las
tInputPath":"abfss://raw@shivghaie.dfs.core.windows.net/sales/"}}
Current End Offsets: {CloudFilesSource[abfss://raw@shivghaie.dfs.core.windows.net/sales/]: {"seqNum":9,"s
ourceVersion":3,"lastBackfillStartTimeMs":1756198538801,"lastBackfillFinishTimeMs":1756198540875,"lastInp
utPath":"abfss://raw@shivghaie.dfs.core.windows.net/sales/"}}

Current State: ACTIVE
```



Just now (14s) 9 SQL

```sql
%sql
SELECT timestamp, level, message
FROM cap2_shiv_catalog.cap2_shiv_schema.event_log
WHERE level = 'ERROR'
ORDER BY timestamp DESC
LIMIT 20
```

(3) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [timestamp: timestamp, level: string ... 1 more field]

Table

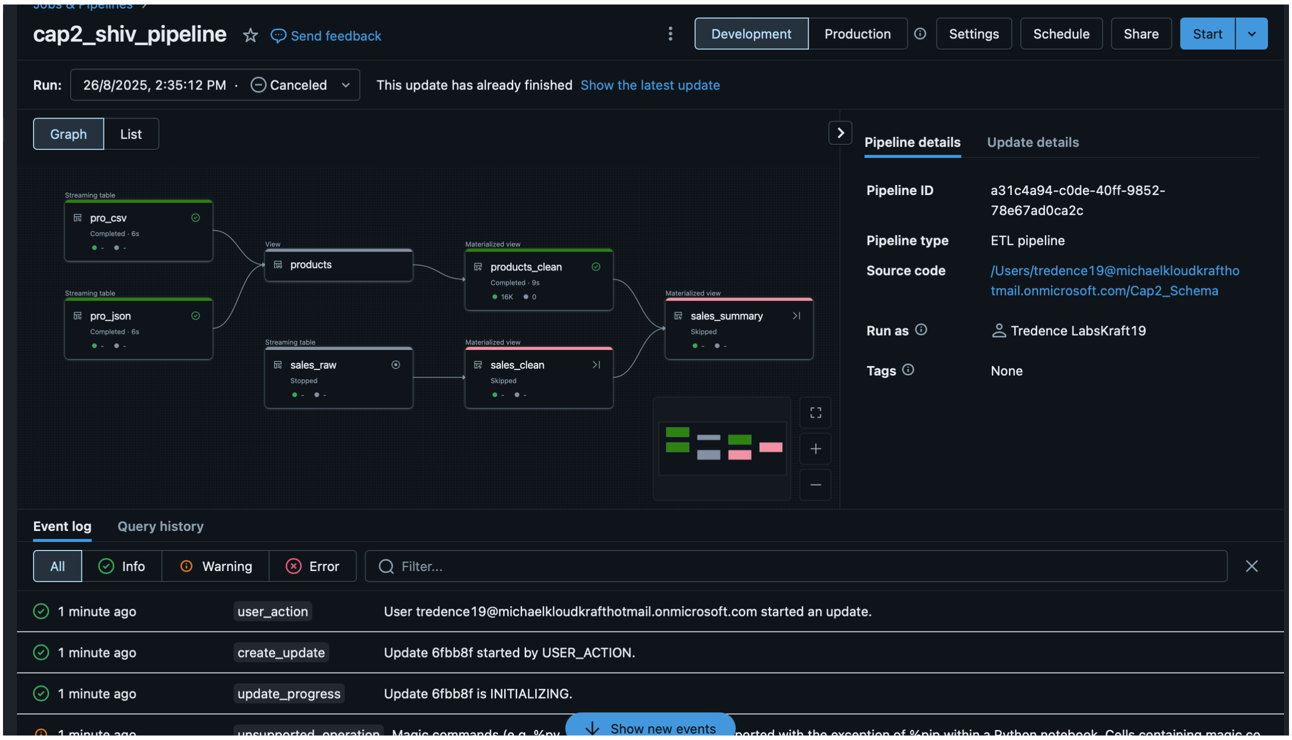
	timestamp	level	message
1	2025-08-26T09:22:42.079+00:00	ERROR	Update 6c5134 has failed while trying to update table 'cap2_shiv_catalog.cap2_shiv_schema.sales_raw'.
2	2025-08-26T09:20:12.356+00:00	ERROR	Update 8d35e5 has failed while trying to update table 'cap2_shiv_catalog.cap2_shiv_schema.sales_raw'.
3	2025-08-26T09:18:26.966+00:00	ERROR	Update 76a811 has failed while trying to update table 'cap2_shiv_catalog.cap2_shiv_schema.sales_raw'.
4	2025-08-26T08:00:31.687+00:00	ERROR	Update 497569 is FAILED.
5	2025-08-26T07:34:15.380+00:00	ERROR	Update 0da085 is FAILED.

Corrupted File

sales_day_1

sale_id	product_id	region	store_id	quantity	sales_amount	sale_date	Demo
1	157	APAC	Store_31	16	1289.03	2025-08-11 21:20:34	Fvfv
2	65	APAC	Store_9	8	1301.48	2025-08-12 02:40:42	Dfvdfvdsav
3	416	LATAM	Store_25	19	3047.56	2025-08-10 00:09:00	Fvdsfav
4	501	US	Store_29	18	1955.74	2025-07-28 09:49:56	Via
5	244	MEA	Store_6	20	2362.98	2025-08-15 00:28:03	Vsdcvads
6	240	EU	Store_22	8	1516.96	2025-08-25 05:58:18	
7	854	EU	Store_24	10	178.92	2025-07-28 09:55:39	
8	69	US	Store_9	7	246.4	2025-08-10 22:41:18	
9	654	EU	Store_3	14	138.23	2025-08-02 05:50:22	
10	277	US	Store_8	15	145.13	2025-08-15 19:05:21	22
11	584	LATAM	Store_4	13	2516.03	2025-08-08 13:20:55	
12	432	APAC	Store_1	13	241.39	2025-07-27 22:24:44	-11
13	797	US	Store_25	11	1914.19	2025-08-05 00:57:54	
14	64	MEA	Store_32	15	2545.81	2025-08-09 06:41:32	Wfwdcvfsad
15	256	US	Store_15	18	1482.75	2025-07-31 09:45:00	
16	924	LATAM	Store_14	7	670.92	2025-08-17 22:33:58	646
17	245	US	Store_38	10	246.84	2025-07-31 15:43:06	
18	779	LATAM	Store_23	6	174.22	2025-07-29 16:52:39	
19	792	MEA	Store_13	7	435.92	2025-08-05 01:06:39	
20	404	APAC	Store_8	10	1822.53	2025-08-05 03:28:00	

Pipeline output with Corrupted File



Automate Error Handling

(i) Bronze Layer

```
import dlt
from pyspark.sql.functions import *
# Ingest raw sales data (multiple daily CSV files in raw/sales/)
@dlt.table(
  name="sales_raw",
  comment="Raw sales data ingested from CSV files",
  table_properties={"quality": "bronze"}
)
def sales_raw():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .option("cloudFiles.inferColumnTypes", "true")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
        .option("cloudFiles.schemaLocation", "abfss://raw@shivghaie.dfs.core.windows.net/
_schemas/sales/")
        .load("abfss://raw@shivghaie.dfs.core.windows.net/sales/")
        # Force sale_id to string regardless of source type
        .withColumn("sale_id", col("sale_id").cast("string"))
    )


# Products CSV
@dlt.table(
  name="pro_csv",
  comment="Raw product data from CSV",
  table_properties={"quality": "bronze"}
)
def pro_csv():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .load("abfss://raw@shivghaie.dfs.core.windows.net/products/")
    )
# Products JSON
@dlt.table(
  name="pro_json",
  comment="Raw product data from JSON",
  table_properties={"quality": "bronze"}
)
def pro_json():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
```

```
        .load("abfss://raw@shivghaie.dfs.core.windows.net/products/")
    )
# Unified Products (CSV + JSON)
@dlt.view(name="products")
def products():
    return dlt.read("pro_csv").unionByName(dlt.read("pro_json"), allowMissingColumns=True)
```

(ii) Silver layer

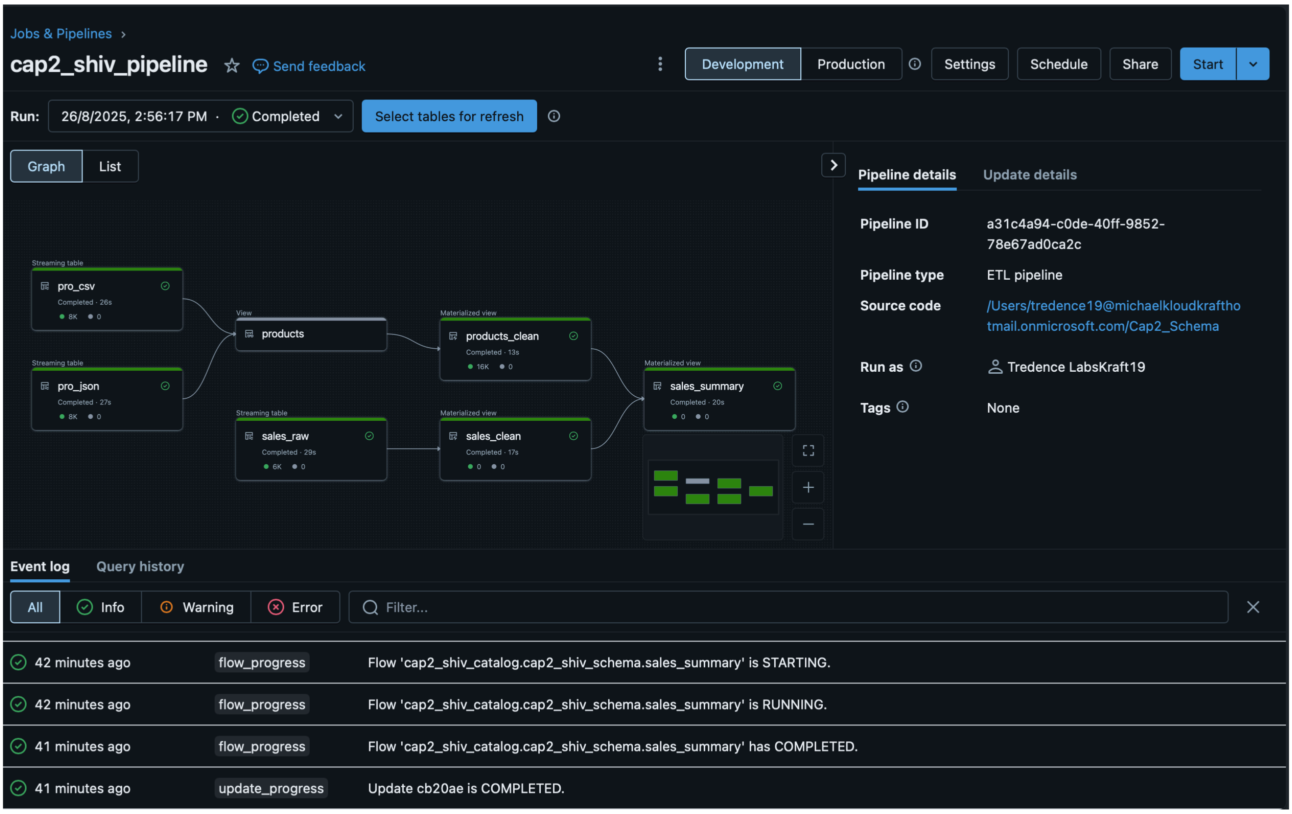
```
# Clean Sales
@dlt.table(
  name="sales_clean",
  comment="Cleaned sales data with enforced schema",
  table_properties={"quality": "silver"}
)
@dlt.expect("valid_sales_amount", "sales_amount >= 0")
@dlt.expect("valid_quantity", "quantity > 0")
def sales_clean():
    return (
        dlt.read("sales_raw")
        .withColumn("sale_id", col("sale_id").cast("string"))
        .withColumn("product_id", col("product_id").cast("string"))
        .withColumn("region", col("region").cast("string"))
        .withColumn("store_id", col("store_id").cast("string"))
        .withColumn("quantity", col("quantity").cast("int"))
        .withColumn("sales_amount", col("sales_amount").cast("double"))
        .withColumn("sale_date", to_date(col("sale_date"), "yyyy-MM-dd"))
        .na.drop()
    )
@dlt.table(
  name="products_clean",
  comment="Cleaned products data with schema evolution support",
  table_properties={"quality": "silver"}
)
def products_clean():
    return (
        dlt.read("products")
        .withColumn("product_id", col("product_id").cast("string"))
        .withColumn("product_name", col("product_name").cast("string"))
        .withColumn("category", col("category").cast("string"))
        .withColumn("price", col("price").cast("double"))
        .withColumn("discount_rate", col("discount_rate").cast("double"))
    )
```

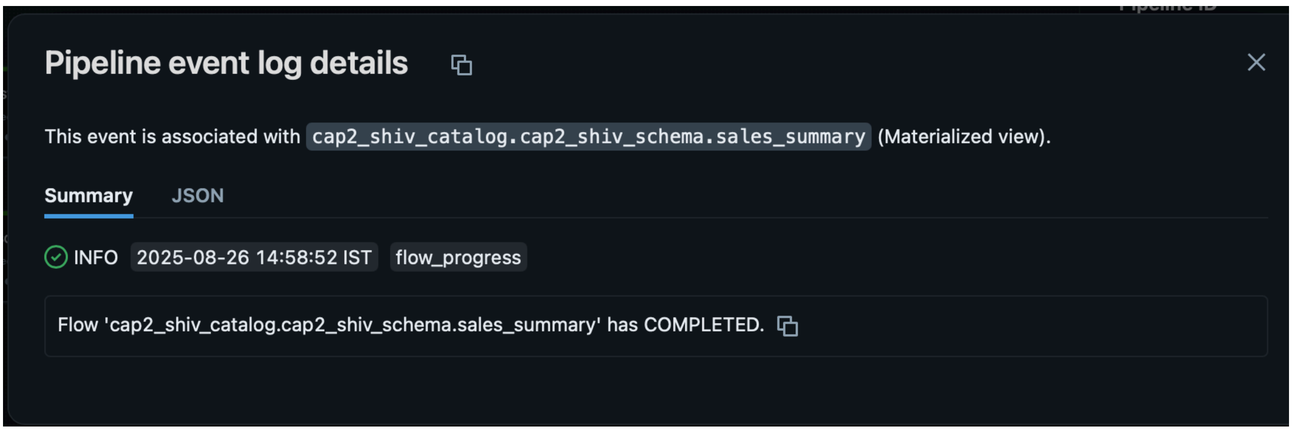
(iii) Gold Layer

```
# Join sales + products and calculate KPIs
@dlt.table(
  name="sales_summary",
```

```
  comment="Daily sales summary by product and region",
  table_properties={"quality": "gold"}
)
def sales_summary():
    sales = dlt.read("sales_clean")
    products = dlt.read("products_clean")
    return (
        sales.join(products, "product_id", "left")
        .groupBy("region", "sale_date", "product_id", "product_name", "category")
        .agg(
            sum("sales_amount").alias("daily_revenue"),
            sum("quantity").alias("units_sold"),
            avg("price").alias("avg_price"),
            avg("discount_rate").alias("avg_discount")
        )
    )
```

Corrected Output





Just now (5s)

9

SQL

```sql
%sql
-- Check if new rows landed in silver (clean data)
SELECT *
FROM cap2_shiv_catalog.cap2_shiv_schema.sales_clean
WHERE sale_date = '2025-08-26';
--Check Gold aggregation
SELECT *
FROM cap2_shiv_catalog.cap2_shiv_schema.sales_summary
WHERE sale_date = '2025-08-26';
--Check event log again (should now show success)
SELECT timestamp, level, message
FROM cap2_shiv_catalog.cap2_shiv_schema.event_log
ORDER BY timestamp DESC
LIMIT 20;
```

(1) Spark Jobs

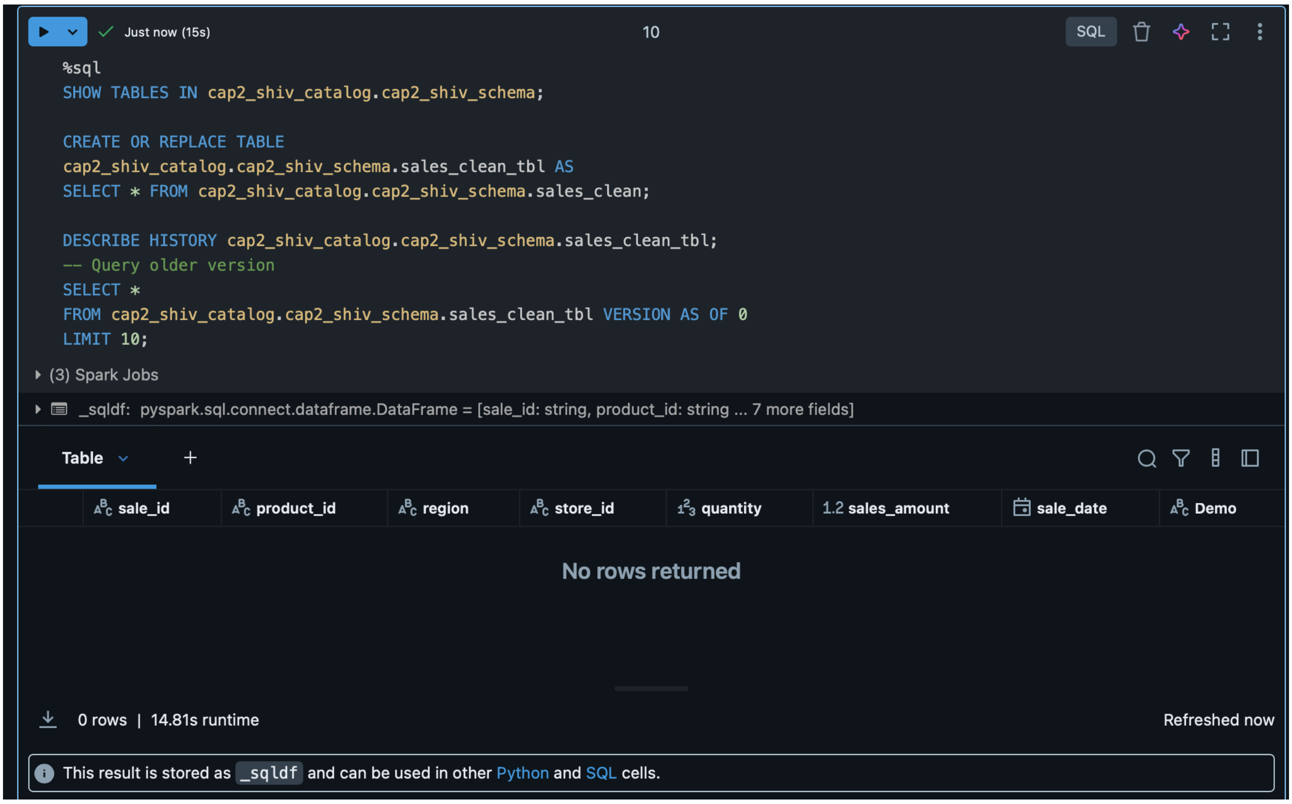
_sqldf: pyspark.sql.connect.dataframe.DataFrame = [timestamp: timestamp, level: string ... 1 more field]

Table

	timestamp	level	message
1	2025-08-26T09:28:54.383+00:00	INFO	Update cb20ae is COMPLETED.
2	2025-08-26T09:28:54.382+00:00	METRICS	Reported flow time metrics for flowName: 'pipelines.flowTimeMetrics.missingFlowName'.
3	2025-08-26T09:28:54.380+00:00	METRICS	Reported flow time metrics for flowName: 'cap2_shiv_catalog.cap2_shiv_schema.products_clean'.
4	2025-08-26T09:28:54.378+00:00	METRICS	Reported flow time metrics for flowName: 'cap2_shiv_catalog.cap2_shiv_schema.sales_summary'.
5	2025-08-26T09:28:52.454+00:00	INFO	Flow 'cap2_shiv_catalog.cap2_shiv_schema.sales_summary' has COMPLETED.
6	2025-08-26T09:28:50.194+00:00	METRICS	Reported cluster resources metrics.
7	2025-08-26T09:28:35.763+00:00	METRICS	> `cap2_shiv_catalog`.`cap2_shiv_schema`.`__materialization_mat_a31c4a94_c0de_40ff_9852_78e67ad0ca2c
8	2025-08-26T09:28:32.973+00:00	INFO	Flow 'cap2_shiv_catalog.cap2_shiv_schema.sales_summary' is RUNNING.
9	2025-08-26T09:28:32.941+00:00	INFO	Flow 'cap2_shiv_catalog.cap2_shiv_schema.sales_summary' is STARTING.
10	2025-08-26T09:28:32.937+00:00	INFO	> Flow 'cap2_shiv_catalog.cap2_shiv_schema.sales_summary' has been planned in DLT to be executed as COMPL

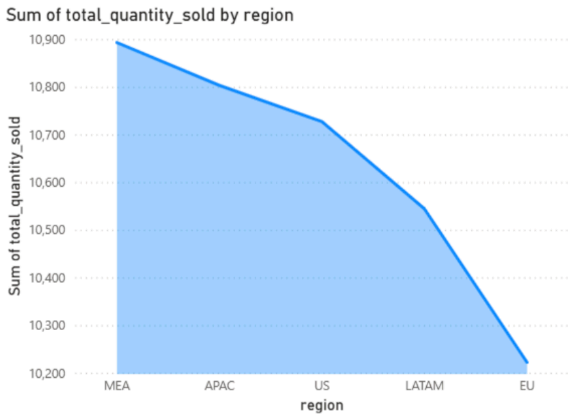
Time travel

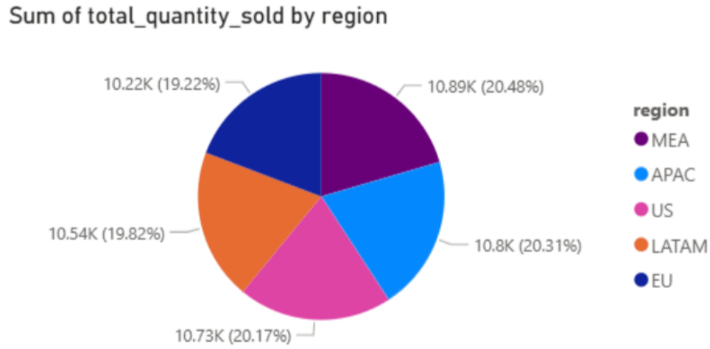
```
%sql
SHOW TABLES IN cap2_shiv_catalog.cap2_shiv_schema;

CREATE OR REPLACE TABLE
cap2_shiv_catalog.cap2_shiv_schema.sales_clean_tbl AS
SELECT * FROM cap2_shiv_catalog.cap2_shiv_schema.sales_clean;

DESCRIBE HISTORY cap2_shiv_catalog.cap2_shiv_schema.sales_clean_tbl;
-- Query older version
SELECT *
FROM cap2_shiv_catalog.cap2_shiv_schema.sales_clean_tbl VERSION AS OF 0
LIMIT 10;
```

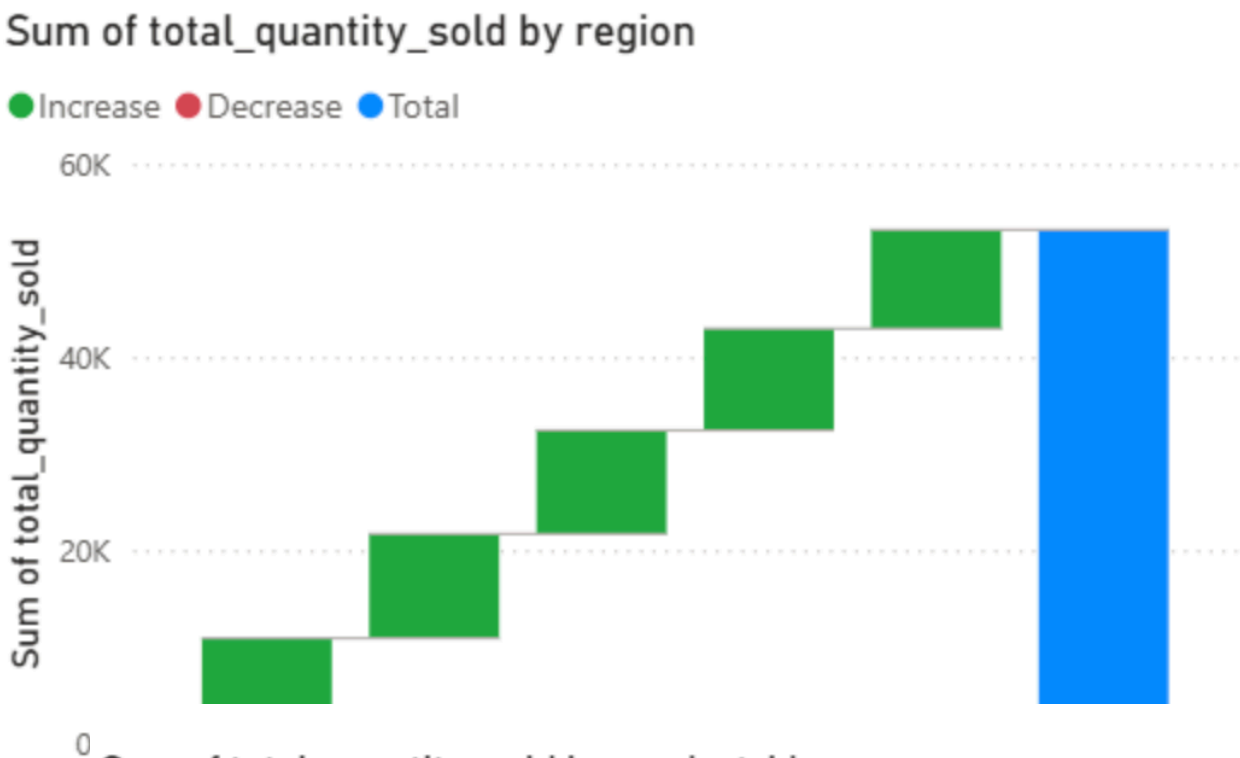


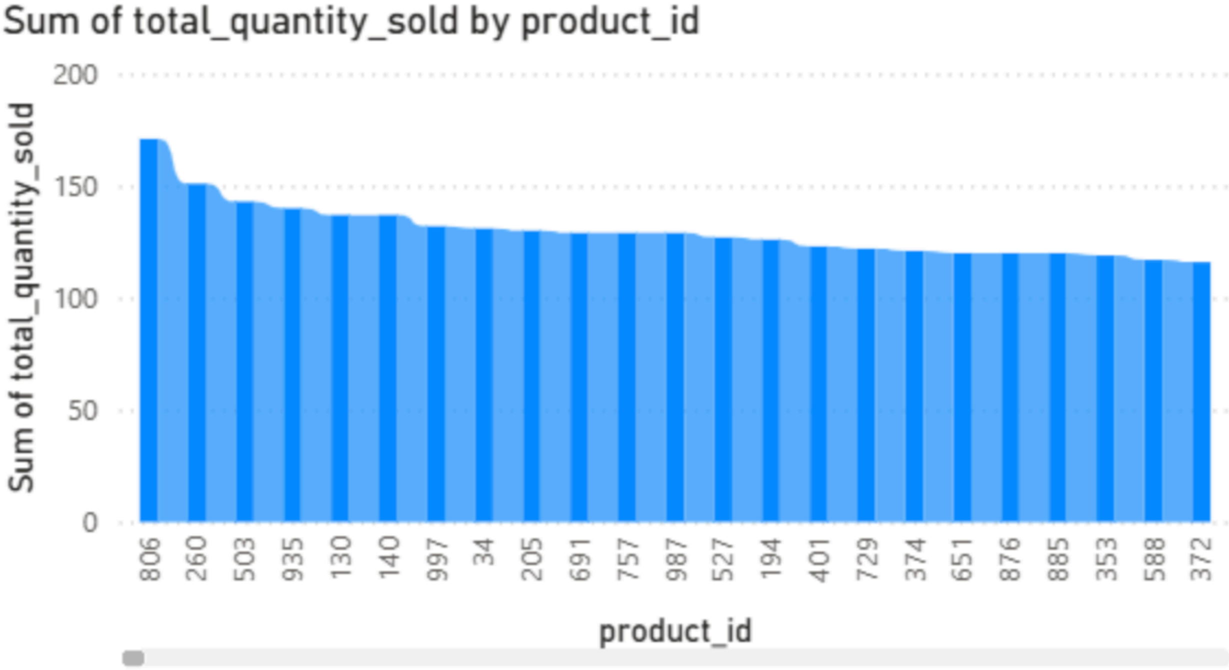
Creation of Automation Workflow Job

Power BI


Sum of total_quantity_sold by region
10,900
10,800
10,700
10,600
10,500
10,400
10,300
10,200
Sum of total_quantity_sold
MEA
APAC
US
LATAM
EU
region


Sum of total_quantity_sold by region
10.22K (19.22%)
10.89K (20.48%)
10.54K (19.82%)
10.8K (20.31%)
10.73K (20.17%)
region
MEA
APAC
US
LATAM
EU


Sum of total_quantity_sold by region
Increase
Decrease
Total
60K
40K
20K
0
Sum of total_quantity_sold


Sum of total_quantity_sold by product_id
200
150
100
50
0
Sum of total_quantity_sold
806
260
503
935
130
140
997
34
205
691
757
987
527
194
401
729
374
651
876
885
353
588
372
product_id